PROJECT THEORY GUIDE

Smart Outcome Predictor

A clear and visual guide to ensemble learning for student course completion and final score prediction.

This guide explains the full project in simple language: dataset preparation, bagging, boosting, voting, stacking, metrics, formulas, model comparison, and safe deployment.

Classification | Regression | Bagging | Boosting | Stacking

START HERE

How to use this guide

Read the pages in order for the full story, or use the map below when revising one topic.

What this project does

It predicts two student outcomes from activity and demographic data:

- **Completion status** — a classification problem with 0 or 1 output.
- **Final score** — a regression problem with a numeric output from 0 to 100.

Why ensembles?

Single trees are easy to understand, but may be unstable. Ensemble methods combine many models, so they can give more reliable predictions.

MAIN IDEA Different models can cover each other's mistakes.

Guide map

Part	What you will learn	Page
1	Problem, targets and full workflow	3
2	Dataset understanding and preprocessing	4–5
3	Decision trees, bagging and random forests	6
4	Boosting: AdaBoost, Gradient Boosting, LightGBM, XGBoost	7–9
5	Voting and stacking ensembles	10
6	Classification and regression evaluation	11–12
7	Tuning, comparison, deployment and final revision sheet	13–15

Simple rule: do not only chase the highest score. A useful model should also be fair, stable, understandable enough, and fast enough for the real task.

PART 1 • FOUNDATION

The problem and the big picture

Before choosing an algorithm, first understand what the prediction means and how the data will move through the project.

TASK A

Course completion

Question: Will a student complete the online course?

Target: **completion_status**

Output: **0** = not completed, **1** = completed

This is a **binary classification** task.

TASK B

Final performance score

Question: What score may the student get?

Target: **final_score**

Output: a number, normally from **0 to 100**

This is a **regression** task.

End-to-end project workflow

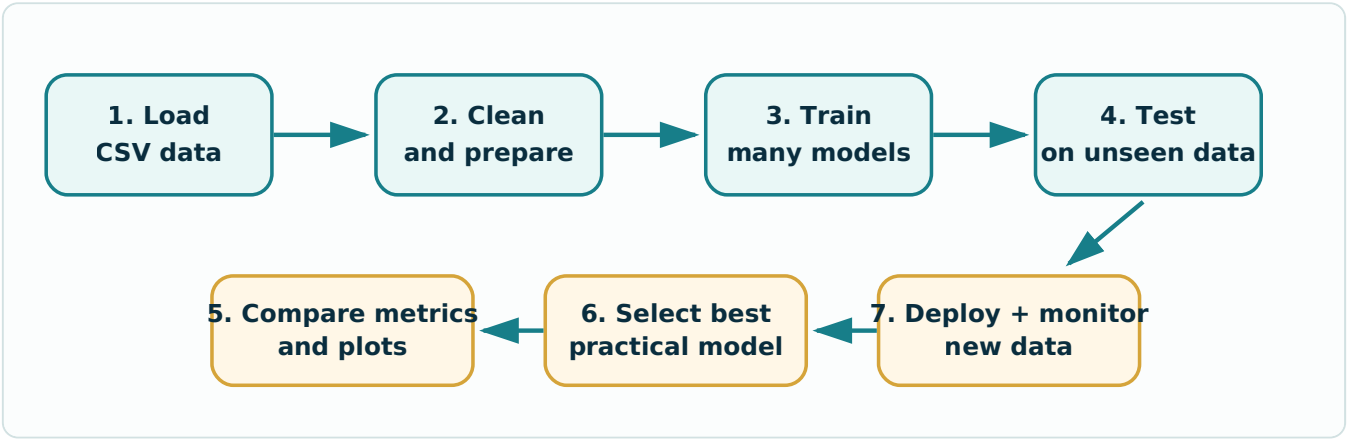


Figure 1. The same clean, held-out test data is used to compare every candidate model fairly.

Important: a prediction is an estimate, not a promise. It should support a teacher or platform team, not automatically make a high-impact decision about a student.

PART 2 • DATA

Dataset understanding

Good models start with good data. The supplied file has 5,200 rows and 19 columns: student profile, course details, engagement behaviour, and two target columns.

Column groups in the supplied dataset

Group	Examples	Why it can help
Student profile	age, country_region, education_background	May show different learning situations or access patterns.
Course context	device_type, course_level, course_category, course_start_date	Describes the learning environment and course type.
Engagement	sessions, time_spent_hours, videos_watched, forum_posts	Shows how a student is interacting with course material.
Assessment activity	quiz_attempts, assignments_submitted, avg_quiz_score, attendance_rate	Usually strong signals of progress and performance.
Targets	completion_status, final_score	The values the models must predict.

Missing values

In this dataset, some numeric columns have missing values, for example time spent, average quiz score and attendance rate.

Instead of deleting many rows, fill numeric missing values with the **median**. Median is less affected by very high or very low values.

```
median = middle value after sorting the
                numbers
```

Identifiers and leakage

student_id is an identifier, not a real learning behaviour. It should be dropped.

Do not use **final_score** to predict completion, or **completion_status** to predict final score. This would leak the answer into input data.

Data leakage: happens when training gets information it would not have at prediction time. It can make test results look much better than reality.

Feature engineering from a date

The raw **course_start_date** can be changed into simple useful features, such as **start_month** and **start_day**. After that, the old date text can be removed.

```
course_start_date → start_month, start_day
```

PART 2 • PREPARATION

Preprocessing and train-test split

Preprocessing makes the raw columns usable for algorithms. The important point is to learn every preprocessing rule from training data only.

Train-test split

80%

Training data

Used to fit preprocessing steps and train each model.

20%

Test data

Kept unseen until the final evaluation. It estimates real-world performance.

For the classification target, use a **stratified split**. This keeps about the same proportion of completed and not-completed students in both parts.

```
Train set = 80%    |    Test set = 20%    |    random_state = 42
```

What happens to each type of feature?

Feature type	Method	Simple reason
Numeric values	Median imputation + standard scaling	Removes missing values and puts features on a similar scale.
Text / categories	Most-frequent imputation + one-hot encoding	Changes category labels into numeric columns without implying an order.
Date	Month / day extraction	Keeps simple time information without giving raw text to the model.

Standard scaling

Scaling is most useful for models that use distances or weights, such as Logistic Regression and Ridge. Tree models normally do not need scaling, but a shared pipeline keeps the notebook simple and safe.

```
z = (x − mean) / standard deviation
```

One-hot encoding example

device_type	new columns	Mobile row
Mobile	Mobile Laptop Tablet	1 0 0

Why a Pipeline? A Pipeline connects preprocessing and model fitting in one object. During cross-validation, every fold learns its filling and encoding rules from its own training fold. This reduces leakage.

PART 3 • BOOTSTRAP AGGREGATING

Decision trees and Bagging

A decision tree learns simple if–then rules. Bagging makes many different trees and combines them.

Decision tree idea

A tree repeatedly asks questions such as:

- Is **attendance_rate > 0.80**?
- Are **assignments_submitted ≥ 4**?
- Is **avg_quiz_score > 65**?

At the last node (leaf), it returns a class or a score.

Strength and weakness

Strength: simple rules and non-linear patterns.

Weakness: a deep tree can memorize training examples. A small change in training data can create a different tree.

How Bagging works

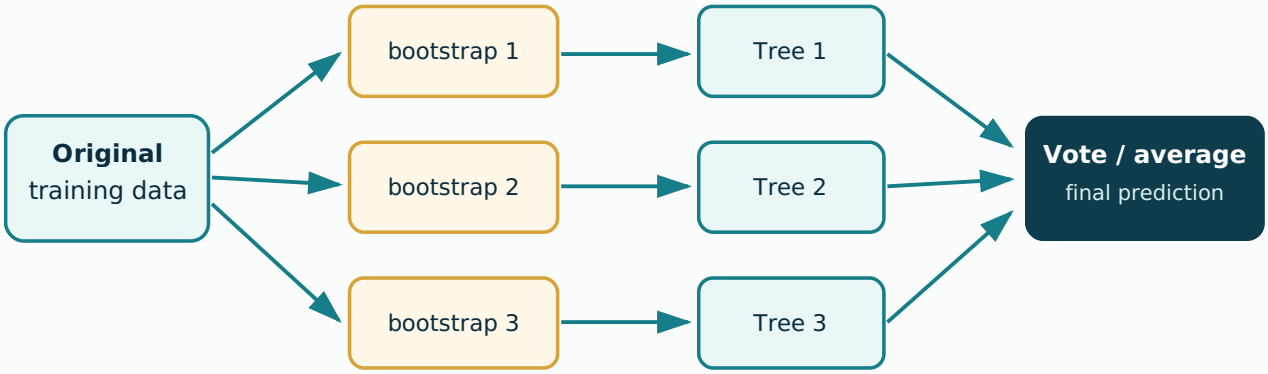


Figure 2. Each bootstrap sample is made by sampling training rows with replacement.

Bagging formulas

```
Classification: final class = majority vote of trees
```

```
Regression: final score = (1/B) × Σ tree prediction
```

Here, **B** is the number of trees. Bagging mainly reduces **variance**. That means the model is less sensitive to one unusual training sample.

Project use: BaggingClassifier and BaggingRegressor use Decision Trees as base models. Their results are compared with a single decision tree.

PART 4 • SEQUENTIAL MODELS

Boosting: learn from earlier errors

Unlike bagging, boosting builds models one after another. Every new learner tries to improve what earlier learners did not explain.

Bagging vs boosting in one view

Point	Bagging	Boosting
Training order	Models train independently.	Models train sequentially.
Main goal	Reduce variance / improve stability.	Reduce bias / correct errors.
Combination	Vote or average.	Weighted sum of weak learners.
Possible risk	May not remove systematic underfitting.	Can overfit noisy rows if tuned badly.

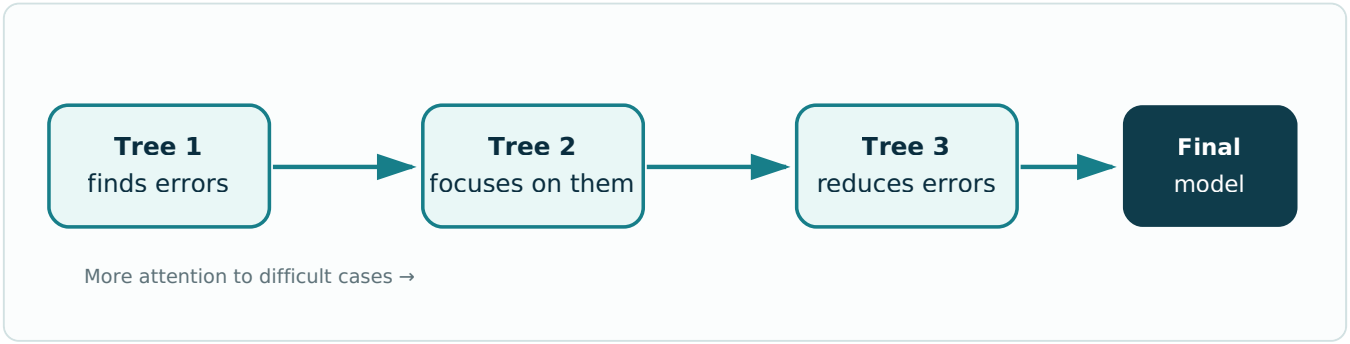
General boosting formula

$$F_m(x) = F_{m-1}(x) + \eta \times h_m(x)$$

$F_m(x)$ is the model after step m. **$h_m(x)$** is the new small tree. **η** (eta) is the learning rate. A smaller eta means smaller, safer steps, but usually needs more trees.

Weak learners

A weak learner is a model that is only a little better than random guessing. In boosting, it is often a shallow tree (a stump has only one split). Many weak learners together can form a strong model.



Key trade-off: more estimators and a high learning rate may fit training data very well, but test performance can fall if the model starts learning noise.

PART 4 • CORE BOOSTING

AdaBoost and Gradient Boosting

Both methods use a sequence of trees, but they correct errors in different ways.

ADABOOST

Adaptive Boosting

AdaBoost starts by giving every training row equal weight. After a tree makes predictions, rows that were wrong get a higher weight. The next tree pays more attention to them.

```
new row weight ↑ when the row was
predicted wrong
```

For classification, the final model is a weighted vote. Better trees get more say.

GRADIENT BOOSTING

Fit the remaining error

Gradient Boosting looks at the error left by the current model. The next tree is trained to predict that error (also called residual).

```
residual = true value − current
prediction
```

It can solve classification and regression tasks.

AdaBoost: simple classification formula

$$\text{Final score} = \text{sign}\left(\alpha_1 h_1(x) + \alpha_2 h_2(x) + \ldots + \alpha_m h_m(x) \right)$$

$h_m(x)$ is a weak learner. **α_m** is its importance. A learner with fewer errors normally receives a higher α value.

Gradient Boosting for regression

1. Start with a simple prediction, often the average target value.
2. Calculate residuals: actual score minus predicted score.
3. Train a small tree to predict residuals.
4. Add a part of that tree prediction using learning rate η.
5. Repeat for the selected number of trees.

$$F_m(x) = F_{m-1}(x) + \eta h_m(x), \quad \text{residual } r_i = y_i - F_{m-1}(x_i)$$

Project settings to explain

Parameter	Meaning	What happens if it is large?
n_estimators	Number of sequential trees	More learning and more time; can overfit.
learning_rate	How much each tree changes the model	Faster learning; may make unstable jumps.
max_depth	How deep each tree can grow	Captures more detail; may learn noise.

Practical idea: a common safe start is a smaller learning rate with enough trees. Always check the held-out test or validation result, not training score only.

PART 4 • ADVANCED GRADIENT BOOSTING

LightGBM and XGBoost

These are high-performance gradient boosting libraries. Both are based on trees, but use extra ideas for speed and control.

LIGHTGBM

Fast leaf-wise growth

LightGBM grows the leaf that can reduce loss the most. It can be very fast and powerful, especially on large data.

Useful controls: number of leaves, learning rate, estimators, feature sampling.

Care: leaf-wise growth can overfit small data if leaves are too many.

XGBOOST

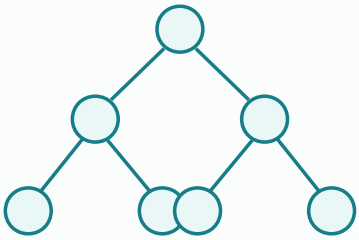
Regularized boosting

XGBoost adds regularization to control tree complexity. It also supports row and column sampling, which can make the model more robust.

Useful controls: max_depth, subsample, colsample_bytree, learning rate, estimators.

How their tree growth differs

Level-wise idea (traditional)



Leaf-wise idea (LightGBM)



LightGBM often grows where it expects the largest loss reduction. This can be efficient but needs sensible limits.

XGBoost objective (main idea)

```
Objective = training loss + regularization penalty
```

The first term asks: How wrong are predictions? The regularization term asks: Is the tree model becoming too complex? The balance helps reduce overfitting.

$$Obj = \Sigma\ loss(y_i, \hat{y}_i) + \Sigma\ \Omega(tree_k)$$

PART 5 • COMBINING DIFFERENT MODELS

Voting and Stacking

Voting is the simpler combination method. Stacking uses a second-level model to learn how to combine base predictions.

VOTING

Hard voting

Each classifier outputs a class. The class with most votes wins.

```
final class = mode(model votes)
```

Soft voting

Each classifier outputs probability. Probabilities are averaged, then the highest class is selected.

```
P(class 1) = average of model
        probabilities
```

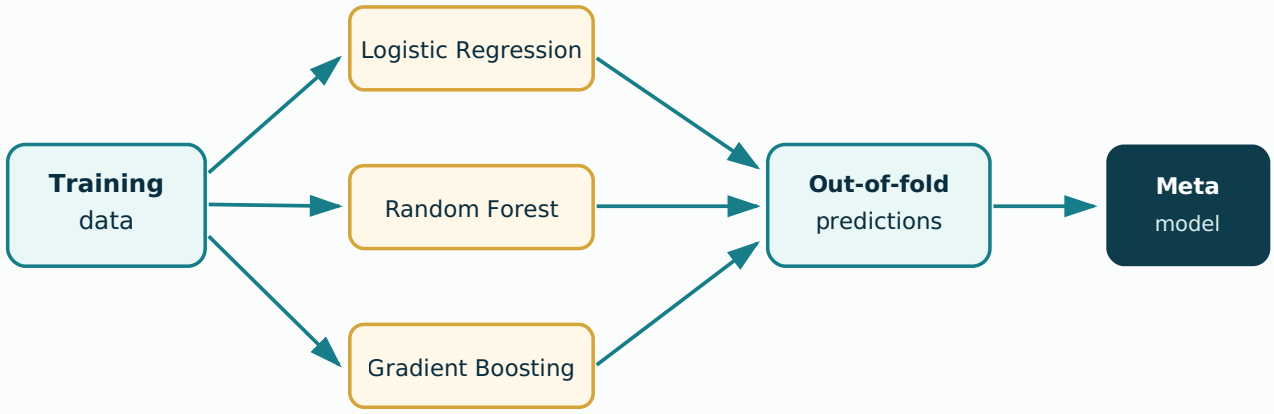
STACKING

Meta-learning

Base models make predictions. A meta-model gets these predictions as its input and learns the best final combination.

In this project, Logistic Regression is the classifier meta-model and Ridge is the regression meta-model.

Stacking workflow with cross-validation



The meta-model learns from out-of-fold predictions, not predictions made on the same rows used to fit the base model.

Why cross-validation matters in stacking: if base models make predictions on their own training rows, the meta-model may see overly optimistic predictions and overfit.

PART 6 • COMPLETION PREDICTION

Classification evaluation

A model must be evaluated from more than one angle. Accuracy alone can hide important mistakes.

Confusion matrix

	Actually completed (1)	Actually not completed (0)
Predicted completed (1)	**True Positive (TP)** Correctly predicts completion.	**False Positive (FP)** Predicts completion when student does not complete.
Predicted not completed (0)	**False Negative (FN)** Misses a student who would complete.	**True Negative (TN)** Correctly predicts non-completion.

Main formulas

```
Accuracy = (TP + TN) / (TP + TN + FP + FN)
```

```
Precision = TP / (TP + FP)
```

```
Recall = TP / (TP + FN)
```

```
F1 = 2 × (Precision × Recall) / (Precision + Recall)
```

Precision

When model says completed, how often is it right? High precision means fewer false positives.

Recall

Out of all students who completed, how many did model find? High recall means fewer false negatives.

ROC-AUC

ROC curve plots **True Positive Rate** against **False Positive Rate** at many probability thresholds. AUC is the area under that curve. A higher AUC means the model is generally better at separating the two classes.

```
AUC ≈ 0.5: random separation    |    AUC = 1.0: perfect separation
```

For this project: F1-score is a good main comparison metric because it balances precision and recall. ROC-AUC is also useful when models provide probabilities.

PART 6 • FINAL-SCORE PREDICTION

Regression evaluation

Regression predictions are numbers, so we measure how far predicted scores are from real scores.

Residual

A residual is the error for one student.

$$\text{residual}_i = \text{actual score } y_i - \text{predicted score } \hat{y}_i$$

Actual	Predicted	Residual
82	75	+7

Main formulas

$$\text{MAE} = (1/n) \times \Sigma\ |y_i - \hat{y}_i|$$

Mean Absolute Error tells the average prediction miss in score points. It is easy to explain: MAE of 8 means predictions are off by about 8 marks on average.

$$\text{RMSE} = \sqrt{[(1/n) \times \Sigma\ (y_i - \hat{y}_i)^2]}$$

Root Mean Squared Error gives extra penalty to large errors. A few very bad score predictions will increase RMSE strongly.

$$R^2 = 1 - [\Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2]$$

R-square compares the model with a basic predictor that always predicts the mean score. Higher is better. R^2 near 1 is very good; R^2 near 0 means little improvement over the mean; negative R^2 means worse than the mean.

> **Which metric should be used?** Report all three. Use MAE for easy business explanation, RMSE when large mistakes are especially bad, and R^2 to understand how much score variation the model explains.

PART 7 • FAIR COMPARISON

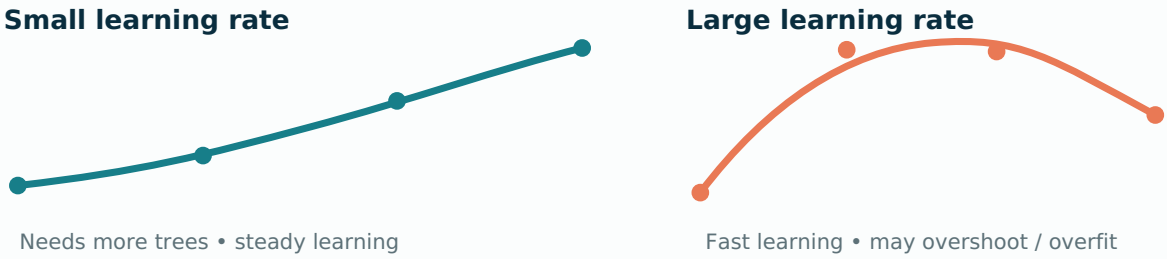
Choosing and tuning models

Model selection should be systematic. First compare a simple baseline, then try ensembles, then choose a model based on performance and practical needs.

Fair comparison checklist

- ✓ Use same train/test split for all models.
- ✓ Use same feature preparation pipeline.
- ✓ Keep the test set unseen while tuning.
- ✓ Compare more than one metric.
- ✓ Measure training time if speed matters.
- ✓ Prefer the simpler model when results are very close.

Learning rate and estimators



Bias and variance

Situation	What it looks like	Possible help
High bias / underfitting	Training and test results both weak.	Use richer features, more trees, or a more flexible model.
High variance / overfitting	Training result high, test result much lower.	Limit depth/leaves, use smaller learning rate, sampling, regularization, or bagging.

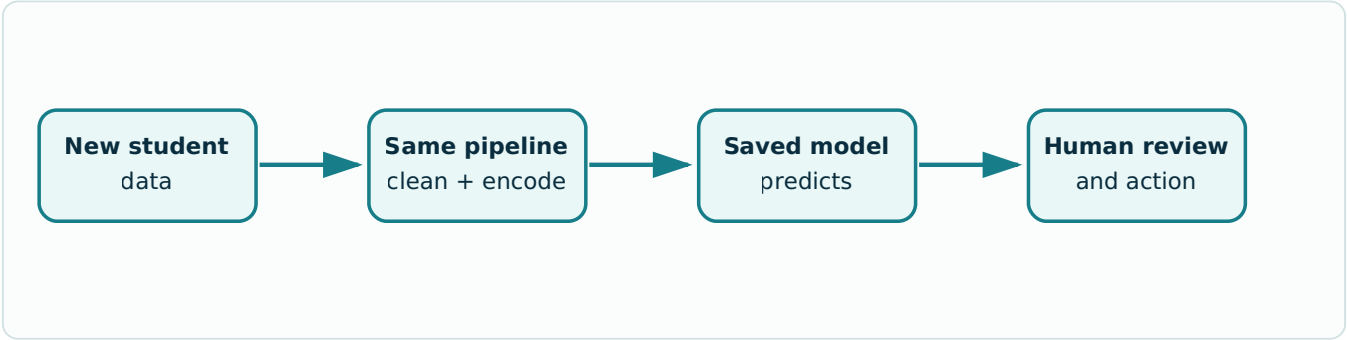
Best model is not always the most complex model. If two models have nearly same test score, prefer the one that trains faster, is easier to keep stable, and gives understandable outputs.

PART 7 • MAKE IT USEFUL SAFELY

Deployment, ethics and monitoring

A model is useful only when it can safely work with new data. Deployment should include the full preprocessing pipeline, not the trained estimator alone.

Simple deployment workflow



Save the whole pipeline

Save the fitted preprocessing and model together. If only the final tree model is saved, new text categories or missing values may not be handled in the same way.

```
saved object = preprocessing steps + trained model
```

Important responsible-use checks

Privacy

Use only necessary student information. Protect file access and do not expose personal data in reports.

Fairness

Check whether results are much worse for groups such as region, device type, or education background.

Human support

Use predictions to offer support, reminders or resources. Do not treat a low prediction as a fixed label for a student.

Monitoring

Over time, check for changed behaviour, missing data, lower accuracy, and new categories. Retrain when needed.

Model drift: student behaviour can change after course design changes, new devices, or new policies. A model that was good last year may become less reliable later.

PART 7 • KEY TAKEAWAYS

Final revision sheet

Use this one-page summary before presenting the project or writing the final report.

Important words

Term	Easy meaning
Ensemble	Many models combined into one final prediction.
Bagging	Train many models on bootstrap samples; vote or average their outputs.
Boosting	Train small models in sequence; later ones improve earlier errors.
Weak learner	A simple model, often a shallow decision tree.
Learning rate	How strongly each new boosting tree changes the current model.
Voting	Combine different model outputs by class vote or average probability.
Stacking	Train a meta-model to combine base-model predictions.
Leakage	Information from target or test data reaches training by mistake.

Project answer flow

1. Prepare

Clean values, encode categories, create date features, drop IDs and targets from X.

2. Model

Train baseline, bagging, boosting, voting and stacking models.

3. Decide

Compare proper metrics and choose practical model for each target.

Final conclusions

- Use **classification** for completion status and **regression** for final score.
- **Bagging** makes tree predictions more stable by averaging many different trees.
- **Boosting** can improve performance by learning sequentially from errors, but must be tuned carefully.
- **LightGBM** and **XGBoost** are efficient boosted-tree libraries with useful controls for complex data.
- **Voting** is a simple combination method; **stacking** learns the combination with a meta-model.
- Report **Accuracy, Precision, Recall, F1-score and ROC-AUC** for classification, and **MAE, RMSE and R^2** for regression.
- Finish with a fair test comparison, careful deployment, and monitoring of new data.

> **One sentence summary:** The Smart Outcome Predictor uses ensemble learning to combine the strength of many models, helping predict course completion and final student score more reliably than one simple model alone.

Prepared as a companion theory document for the Smart Outcome Predictor notebook. All formulas are shown in simplified form for learning and presentation.